REVISION CHEAT SHEET: APPLICATION ARCHITECTURE

.NET Application Architecture

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

1. What Is Application Architecture?
2. Architecture Is Larger Than a Design Pattern
3. Architecture Begins With Requirements
4. Architecture Should Manage Change
5. Logical Architecture vs Physical Architecture
Diagram 1 — Layers vs Deployment
6. The Simplest Architecture: One Project
7. Monolithic Architecture
8. Benefits of a Monolith
9. Problems With a Poorly Structured Monolith
10. Layered Architecture
11. Traditional Three-Layer Example
12. Problem With Traditional Layering
13. Clean Architecture
Diagram 2 — Clean Architecture Dependency Direction
14. A Practical Four-Project .NET Structure
15. Domain Project
16. Domain Entity Example
17. Application Project
18. Application Service Example
19. Application Interfaces as Ports
20. Infrastructure Project
21. Infrastructure Repository
22. API / Presentation Project
23. Thin Controller Example
24. Compile-Time Dependencies
25. Runtime Dependencies Are Different
Diagram 3 — Fundraising Clean Architecture Flow
26. Domain vs Application Logic
Domain logic
27. Application Logic
28. Infrastructure Logic
29. Presentation Logic
30. Keep Domain Entities Away From API Contracts
31. Clean Architecture Is Not “Four Projects”
32. Clean Architecture Is Not Mandatory Everywhere
33. Vertical Slice Architecture
34. Horizontal vs Vertical Organization
35. Vertical Slice Does Not Eliminate Architecture
36. Clean Architecture + Vertical Slices
37. Modular Monolith
38. Why a Modular Monolith Is Powerful
39. Module Boundary Example
40. Module Boundaries Should Be Business-Oriented
41. Modular Monolith vs Layered Monolith
42. Microservices Architecture
43. Microservices Are Not “Small APIs”
44. Microservice Boundary Should Reflect Business Cohesion
45. Benefits of Microservices
46. Costs of Microservices
47. When Microservices May Be Justified
48. Start With a Modular Monolith?
49. Database Ownership in a Monolith
50. Database Ownership in Microservices
51. Transactions in a Monolith
52. Transactions Across Microservices
53. Domain Events vs Integration Events
54. Why Integration Events Must Follow Commit
55. Outbox Pattern
56. Integration Flow
57. Eventual Consistency
58. Idempotency

59. CQRS
60. Simple CQRS in Fundraising
61. CQRS Does Not Require MediatR
62. CQRS Does Not Require Separate Databases
63. Query Side Should Be Practical
64. Vertical Slice + CQRS
65. Background Processing Architecture
Diagram 4 — Web + Background Worker
66. Do Not Use Fire-and-Forget Request Tasks
67. In-Memory Queue vs Durable Queue
68. Cross-Cutting Concerns
69. Middleware
70. Decorators / Pipelines
71. Dependency Injection as Composition Root
72. Configuration Architecture
73. Security Architecture
74. Multi-Tenant Architecture
75. Do Not Put Tenant State in Singleton
76. Scalability
77. Stateless API Instances
78. Caching Architecture
79. Performance and Architecture
80. Chatty Microservices
81. Resilience Architecture
82. Observability
83. Correlation
84. Aspire and Distributed .NET Architecture
85. Testing Architecture
86. Architecture Dependency Tests
87. Domain Unit Test Example
88. Application Unit Test
89. Integration Test
90. Common Architecture Mistake — Controller Owns Everything
91. Common Mistake — Repository Owns Business Rules
92. Common Mistake — Domain Depends on Infrastructure
93. Common Mistake — Infrastructure Types Leak Everywhere
94. Common Mistake — Abstraction for Every Class
95. Common Mistake — Repository Around Every EF Method
96. Common Mistake — Microservices by Database Table
97. Distributed Monolith
98. Common Mista Autonomous Services
99. Common Mistake — CQRS Everywhere
100. Common Mistake — Domain Events for Everything
101. Common Mistake — One Shared “Common” Project
102. Common Mistake — Generic Base Classes for Architecture
103. Common Mistake — Architecture Astronautics
104. Common Mistake — No Architecture at All
105. Recommended Fundraising Platform Starting Architecture
106. Domain Organization
107. Application Vertical Slices
108. Infrastructure Organization
109. API Organization
110. End-to-End Create Gift Flow
111. Complete Transaction Example
112. Background Outbox Architecture
Diagram 5 — Complete Fundraising Architecture
113. Architecture Decision Guide
114. Architecture and Performance
115. Architecture and Memory
116. Architecture and Thread Safety
117. Architecture and Security
118. Architecture and Failure
119. Architecture Documentation
120. Architecture Decision Records
121. Practical Exercise 1 — Layer Identification
122. Practical Exercise 2 — Find Dependency Violation
123. Practical Exercise 3 — Find Business Rule
124. Practical Exercise 4 — Application Orchestration
125. Practical Exercise 5 — Vertical Slice
126. Practical Exercise 6 — Modular Monolith
127. Practical Exercise 7 — Microservice Decision
128. Practical Exercise 8 — CQRS
129. Practical Exercise 9 — Outbox
130. Practical Exercise 10 — Security Boundary
131. Practical Exercise 11 — Distributed Failure
132. Practical Exercise 12 — Architecture Review
133. Exactly 25 Interview Questions
1. What is software/application architecture?

2. What is the difference between a layer and a tier?
3. What is a monolithic architecture?
4. What is layered architecture?
5. What is Clean Architecture?
6. What is the dependency rule in Clean Architecture?
7. What belongs in the Domain layer?
8. What belongs in the Application layer?
9. What belongs in Infrastructure?
10. Why should domain entities not normally be returned directly from Web APIs?
11. What is Vertical Slice Architecture?
12. What is a Modular Monolith?
13. What is the difference between a modular monolith and microservices?
14. What is a bounded context?
15. Why is a microservice per database table usually a bad design?
16. What is CQRS?
17. What is the diffe query?
18. What is the difference between a domain event and an integration event?
19. What is eventual consistency?
20. What is the Outbox pattern and why is it useful?
21. What is a distributed monolith?
22. When should you choose microservices?
23. How do architecture and dependency injection relate?
24. How should you test a Clean Architecture application?
25. What architecture would you choose initially for the Fundraising Platform?
134. Quick Revision
135. Final Architecture Decision Flow
136. Final End-to-End Fundraising Concept Flow

1. What Is Application Architecture?

Application architecture describes the **high-level organization of a software system**: how responsibilities are divided, where business rules live, how components communicate, which dependencies are allowed, how data moves through the application, and how the system is deployed and operated.

It answers questions such as:

- Where should controllers live?
- Where should business rules live?
- Should the Domain project reference EF Core?
- Should repositories be defined in Application or Infrastructure?
- Where should DTOs go?
- Where should HTTP clients live?
- Should a feature be another class, another module, or another microservice?
- How do database transactions fit into the architecture?
- How do background workers interact with the application?
- Where should authorization, logging, caching, and validation happen?
- How can the application change without creating dependencies everywhere?

A good architecture does not exist simply to make the solution look organized. Its purpose is to make important changes **safe, understandable, testable, and economical**.

Microsoft's current .NET architecture guidance emphasizes maintainability, separation of concerns, explicit interfaces, dependency inversion, and placing business logic at the center rather than coupling it directly to infrastructure.

2. Architecture Is Larger Than a Design Pattern

From Topic 33, patterns such as Strategy, Factory, Adapter, and Decorator solve relatively focused design problems.

Architecture works at a larger scale.

For example:

Level	Example question
Language	Should this be a class, record, or struct?
OOP/design	Should this behavior use Strategy?
Component	Should this be an application service?
Architecture	Which layer owns this behavior?
Deployment	Is this one process or multiple services?
System	How do independently deployed services communicate?

A Strategy pattern might exist **inside** the Application layer.

A Repository might exist between Application and Infrastructure.

A message broker might connect two independently deployed bounded contexts.

These are different levels of design.

3. Architecture Begins With Requirements

Do not begin with:

> ⓘ “Should we use microservices?”

Begin with:

> ⓘ “What does the system need to accomplish, and what qualities must it have?”

Architecture is influenced by both **functional requirements** and **non-functional requirements**.

Functional requirements describe capabilities:

- Create a Gift.
- Allocate a Gift to Funds.
- Associate a Gift with a Campaign.
- Generate a receipt.
- Search Constituents.
- Import Gifts.
- Process integration messages.

Non-functional requirements describe qualities:

- Availability
- Performance
- Scalability
- Security
- Maintainability
- Testability
- Auditability
- Reliability
- Recoverability
- Deployment frequency
- Operational complexity

- Data consistency

For example, these two systems may implement exactly the same Gift API:

EXAMPLE / FLOW

```
System A
10 internal users
100 Gifts/day
Single development team
One deployment/month
```

and:

EXAMPLE / FLOW

```
System B
Millions of Gifts/day
24/7 availability
Independent teams
Different scaling profiles
Frequent releases
```

The appropriate architectures might be completely different.

4. Architecture Should Manage Change

A useful architectural question is:

> **What is likely to change independently?**

Examples:

- SQL Server may change.
- Email provider may change.
- Business rules may change frequently.
- API contract may version independently.
- Reporting requirements may grow.
- Payment integration may be replaced.
- Background processing may scale differently from API processing.

Architecture should isolate important changing dimensions without creating unnecessary abstraction everywhere.

This connects directly to:

- SRP
- OCP
- DIP
- Separation of Concerns
- Explicit Dependencies

from Topics 31 and 32.

5. Logical Architecture vs Physical Architecture

This distinction is fundamental.

A **layer** is a logical separation of responsibilities.

A **tier** is a physical deployment boundary.

Microsoft's architecture guidance explicitly distinguishes these concepts: an application can contain several logical layers while still being deployed as a single physical unit. APPLICATION ARCHITECTURE

For example:

EXAMPLE / FLOW
```
Fundraising.Api
Fundraising.Application
Fundraising.Domain
Fundraising.Infrastructure
```

could all ultimately run inside:

EXAMPLE / FLOW
```
one ASP.NET Core process
```

That is logically layered but physically monolithic.

Diagram 1 — Layers vs Deployment

EXAMPLE / FLOW
```
LOGICAL VIEW

API
 ↓
Application
 ↓
Domain
 ↑
Infrastructure implementations


PHYSICAL VIEW

┌──────────────────────────────┐
│ One ASP.NET Core application │
│                              │
│ API                          │
│ Application                  │
│ Domain                       │
│ Infrastructure               │
└──────────────────────────────┘

          ↓
       SQL Server
```

Multiple projects do **not** automatically mean microservices.

6. The Simplest Architecture: One Project

A new ASP.NET Core application can start as one project:

EXAMPLE / FLOW
```
Fundraising.Api
```

containing:

EXAMPLE / FLOW
```
Controllers/
Services/
Models/
```

```
Data/
Program.cs
```

APPLICATION ARCHITECTURE

For a small application, this can be completely reasonable.

Microsoft notes that an all-in-one monolithic application is the simplest deployment model and can work very well for smaller or internal applications.

The problem appears when complexity grows and responsibilities become blurred.

You might eventually see:

EXAMPLE / FLOW

```
Controllers/
  GiftController.cs
  CampaignController.cs

Services/
  GiftService.cs
  CampaignService.cs
  HelperService.cs
  CommonService.cs

Models/
  everything...

Data/
  everything...
```

Now developers struggle to answer:

> ⓘ Which code is allowed to depend on which other code?

Folders provide organization, but they do not necessarily enforce dependency boundaries.

7. Monolithic Architecture

A monolith is an application whose core behavior runs as one deployable application/process.

A monolith can still be:

- Well designed
- Layered
- Modular
- Highly testable
- Horizontally scalable
- Containerized

Monolith does **not** mean:

EXAMPLE / FLOW

```
bad architecture
```

and it does not mean:

EXAMPLE / FLOW

```
one giant class
```

Microsoft explicitly notes that a monolithic deployment can internally contain multiple libraries, layers, and components while remaining one deployment unit.

8. Benefits of a Monolith

A well-designed monolith typically has simpler:

- Deployment
- Debugging
- Local development
- Database transactions
- Refactoring
- End-to-end testing
- Operational monitoring
- Failure handling

A method call remains:

```C#
await giftService.CreateAsync(...);
```

rather than becoming:

```
EXAMPLE / FLOW
HTTP/gRPC call
+
network timeout
+
retry
+
serialization
+
authentication
+
distributed tracing
+
partial failure
```

This simplicity is valuable.

9. Problems With a Poorly Structured Monolith

A poorly designed monolith can become a:

```
EXAMPLE / FLOW
Big Ball of Mud
```

with:

```
EXAMPLE / FLOW
Everything depends on everything.
```

For example:

```
EXAMPLE / FLOW
GiftController
      ↓
CampaignRepository
      ↓
PaymentClient
      ↓
```

```
GiftService
    ↓
DbContext
    ↓
ReceiptService
    ↓
GiftController helper
```

Typical symptoms include:

- Circular dependencies
- Business logic in controllers
- Business rules in repositories
- Direct DbContext access everywhere
- Infrastructure models leaking into API contracts
- Static shared state
- Huge service classes
- Cross-feature coupling

The problem is not monolithic deployment itself.

The problem is lack of internal boundaries.

10. Layered Architecture

A traditional layered application commonly separates:

EXAMPLE / FLOW

```
Presentation
Business Logic
Data Access
```

or:

EXAMPLE / FLOW

```
UI
BLL
DAL
```

Microsoft describes traditional N-layer applications in exactly this general form.

Conceptually:

EXAMPLE / FLOW

```
Controller
    ↓
Business Service
    ↓
Repository
    ↓
Database
```

11. Traditional Three-Layer Example

Presentation

C#

```
[ApiController]
[Route("api/gifts")]
```

```
public sealed class GiftController : ControllerBase
{
    private readonly GiftService _service;

    public GiftController(
        GiftService service)
    {
        _service = service;
    }
}
```


Business

C#

```
public sealed class GiftService
{
    private readonly GiftRepository _repository;

    public GiftService(
        GiftRepository repository)
    {
        _repository = repository;
    }
}
```

Data

C#

```
public sealed class GiftRepository
{
    private readonly FundraisingDbContext _dbContext;

    public GiftRepository(
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }
}
```

Compile-time dependency:

EXAMPLE / FLOW

```
Presentation
     ↓
Business
     ↓
Data
```

12. Problem With Traditional Layering

The important business layer often depends directly on the data-access layer.

That means:

EXAMPLE / FLOW

```
Business
   ↓
EF Core / SQL implementation
```

If the business layer is supposed to contain the most valuable application policy, this dependency direction is undesirable.

Microsoft's guidance highlights this exact disadvantage of traditional layering: compile-time dependencies often run from UI → business → data, causing important business logic to depend on infrastructure details.

Dependency Inversion addresses this.

13. Clean Architecture

Clean Architecture places business and application rules near the center and pushes infrastructure details toward the outside.

Microsoft's .NET architecture documentation groups related approaches such as:

- Hexagonal Architecture
- Ports and Adapters
- Onion Architecture
- Clean Architecture

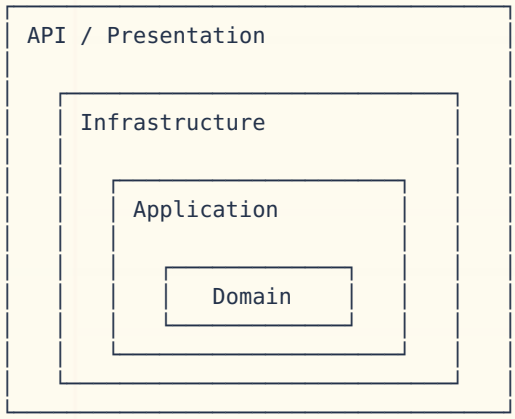
as closely related styles that all emphasize keeping core business logic independent of infrastructure.

The central idea is:

> **Dependencies should point toward the application/domain core.**

Diagram 2 — Clean Architecture Dependency Direction

EXAMPLE / FLOW



Compile-time dependencies point inward.

14. A Practical Four-Project .NET Structure

For the Fundraising Platform, a practical structure could be:

EXAMPLE / FLOW

```
Fundraising.Domain
Fundraising.Application
Fundraising.Infrastructure
Fundraising.Api
```

plus tests:

EXAMPLE / FLOW

```
Fundraising.Domain.Tests
Fundraising.Application.Tests
Fundraising.Infrastructure.Tests
Fundraising.Api.IntegrationTests
```

The exact number of projects is not the principle.

The important part is the **dependency direction**.

15. Domain Project

The Domain project contains the core business concepts and rules.

Example types:

EXAMPLE / FLOW

```
Constituent
Gift
GiftAllocation
Campaign
Fund

GiftStatus
PaymentMethod

Money
GiftReference

Domain events
Domain services
Business exceptions
```

The Domain layer should not need to know:

EXAMPLE / FLOW

```
ASP.NET Core
SQL Server
EF Core
HttpClient
SMTP
Azure Service Bus
Controller
JSON
```

Microsoft's DDD guidance states that the domain model should express business concepts and rules while ignoring persistence and infrastructure details.

16. Domain Entity Example

C#

```
public sealed class Gift
{
    private readonly List<GiftAllocation> _allocations = [];

    public long GiftId { get; private set; }

    public long TenantId { get; }

    public long ConstituentId { get; }
```

```
    public decimal TotalAmount { get; }

    public GiftStatus Status { get; private set; }

    public IReadOnlyCollection<GiftAllocation> Allocations =>
        _allocations;

    private Gift(
        long tenantId,
        long constituentId,
        decimal totalAmount)
    {
        if (tenantId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(tenantId));
        }

        if (constituentId <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(constituentId));
        }

        if (totalAmount <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(totalAmount));
        }

        TenantId = tenantId;
        ConstituentId = constituentId;
        TotalAmount = totalAmount;
        Status = GiftStatus.Pending;
    }

    public static Gift Create(
        long tenantId,
        long constituentId,
        decimal totalAmount)
        =>
        new(
            tenantId,
            constituentId,
            totalAmount);

    public void AddAllocation(
        long fundId,
        decimal amount)
    {
        if (amount <= 0)
        {
            throw new ArgumentOutOfRangeException(
                nameof(amount));
        }

        decimal allocated = _allocations.Sum(x => x.Amount);

        if (allocated + amount > TotalAmount)
        {
            throw new InvalidOperationException(
                "Gift allocations cannot exceed the gift amount.");
        }

        _allocations.Add(
```

```
            new GiftAllocation(
                fundId,
                amount));
    }

    public void Complete()
    {
        if (Status != GiftStatus.Pending)
        {
            throw new InvalidOperationException(
                "Only pending gifts may be completed.");
        }

        if (_allocations.Sum(x => x.Amount) != TotalAmount)
        {
            throw new InvalidOperationException(
                "Gift must be fully allocated before completion.");
        }

        Status = GiftStatus.Completed;
    }
}
```


Notice what is missing:

EXAMPLE / FLOW

```
DbContext
ILogger
IHttpClientFactory
IConfiguration
HttpContext
```

Those are not domain concepts.

17. Application Project

The Application layer defines:

> **What the software should do for a use case.**

Examples:

EXAMPLE / FLOW

```
CreateGift
CompleteGift
RefundGift
SearchConstituents
AssignCampaign
GenerateReceipt
ProcessOutbox
```

Microsoft describes the application layer as coordinating tasks while delegating actual domain rules to domain objects rather than containing domain state or transactional business rules itself.

The Application layer can contain:

- Use-case services
- Command handlers
- Query handlers
- DTOs

- Interfaces/ports
- Validation for application inputs
- Mapping
- Authorization abstractions
- Transaction orchestration
- Domain-event handlers

APPLICATION ARCHITECTURE

18. Application Service Example

C#

```
public sealed class CreateGiftService
{
    private readonly IGiftRepository _giftRepository;
    private readonly IUnitOfWork _unitOfWork;

    public CreateGiftService(
        IGiftRepository giftRepository,
        IUnitOfWork unitOfWork)
    {
        _giftRepository = giftRepository;
        _unitOfWork = unitOfWork;
    }

    public async Task<CreateGiftResult> CreateAsync(
        CreateGiftCommand command,
        CancellationToken cancellationToken)
    {
        Gift gift =
            Gift.Create(
                command.TenantId,
                command.ConstituentId,
                command.TotalAmount);

        foreach (GiftAllocationRequest allocation
                 in command.Allocations)
        {
            gift.AddAllocation(
                allocation.FundId,
                allocation.Amount);
        }

        await _giftRepository.AddAsync(
            gift,
            cancellationToken);

        await _unitOfWork.SaveChangesAsync(
            cancellationToken);

        return new CreateGiftResult(
            gift.GiftId,
            gift.TotalAmount,
            gift.Status);
    }
}
```

Responsibilities:

EXAMPLE / FLOW

```
Receive use-case input
Create/use domain objects
Coordinate repository
```

```
Commit unit of work
Return application result
```


It should not implement:

EXAMPLE / FLOW

```
SQL statements
HTTP status codes
SMTP
JSON formatting
```

19. Application Interfaces as Ports

Clean/Hexagonal architecture often refers to abstractions crossing the core boundary as **ports**.

Examples:

C#

```
public interface IGiftRepository
{
    Task<Gift?> FindAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken);

    Task AddAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

C#

```
public interface IReceiptSender
{
    Task SendAsync(
        ReceiptMessage receipt,
        CancellationToken cancellationToken);
}
```

C#

```
public interface ICurrentUser
{
    long UserId { get; }

    long TenantId { get; }
}
```

The Application layer defines what it needs.

Infrastructure supplies implementations.

That is Dependency Inversion.

20. Infrastructure Project

Infrastructure contains implementations involving technology details:

- EF Core
- SQL Server
- File storage
- Email

- HTTP APIs
- Message brokers
- Distributed caches
- Cloud SDKs
- External payment systems

Microsoft's guidance places EF `DbContext`, migrations, repositories, and infrastructure-specific service implementations in the Infrastructure portion of a Clean Architecture solution.

21. Infrastructure Repository

C#

```
public sealed class GiftRepository :
    IGiftRepository
{
    private readonly FundraisingDbContext _dbContext;

    public GiftRepository(
        FundraisingDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public Task<Gift?> FindAsync(
        long tenantId,
        long giftId,
        CancellationToken cancellationToken)
    {
        return _dbContext.Gifts
            .SingleOrDefaultAsync(
                gift =>
                    gift.TenantId == tenantId
                    &&
                    gift.GiftId == giftId,
                cancellationToken);
    }

    public async Task AddAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        await _dbContext.Gifts.AddAsync(
            gift,
            cancellationToken);
    }
}
```

Infrastructure knows:

EXAMPLE / FLOW

```
EF Core
DbContext
SQL mappings
```

Application only knows:

EXAMPLE / FLOW

```
IGiftRepository
```

22. API / Presentation Project

The API layer is the system's HTTP boundary.

It should own things such as:

- Routes
- Controllers/endpoints
- Model binding
- HTTP status codes
- Authentication/authorization configuration
- Request/response contracts
- Middleware
- API versioning
- OpenAPI
- Exception-to-HTTP translation

It should not become the primary home of business rules.

23. Thin Controller Example

C#

```
[ApiController]
[Route("api/gifts")]
public sealed class GiftController :
    ControllerBase
{
    private readonly CreateGiftService _createGiftService;

    public GiftController(
        CreateGiftService createGiftService)
    {
        _createGiftService = createGiftService;
    }

    [HttpPost]
    public async Task<ActionResult<CreateGiftResponse>>
        Create(
            CreateGiftRequest request,
            CancellationToken cancellationToken)
    {
        CreateGiftCommand command =
            new(
                TenantId:
                    request.TenantId,
                ConstituentId:
                    request.ConstituentId,
                TotalAmount:
                    request.TotalAmount,
                Allocations:
                    request.Allocations);

        CreateGiftResult result =
            await _createGiftService.CreateAsync(
                command,
                cancellationToken);

        return CreatedAtAction(
            nameof(Get),
            new
```

```
            {
                giftId =
                    result.GiftId
            },
            new CreateGiftResponse(
                result.GiftId,
                result.TotalAmount,
                result.Status));
    }

    [HttpGet("{giftId:long}")]
    public IActionResult Get(
        long giftId)
    {
        throw new NotImplementedException();
    }
}
```

The controller translates:

EXAMPLE / FLOW

```
HTTP request
    ↓
Application command
```

and:

EXAMPLE / FLOW

```
Application result
    ↓
HTTP response
```

24. Compile-Time Dependencies

A typical project-reference arrangement could be:

EXAMPLE / FLOW

```
Fundraising.Domain
    →
references no application projects

Fundraising.Application
    →
references Domain

Fundraising.Infrastructure
    →
references Application + Domain

Fundraising.Api
    →
references Application
    →
references Infrastructure only at composition/startup if needed
```

The precise project references vary by implementation, but the critical idea remains:

> **Domain/business policy must not depend on infrastructure implementation details.**

Microsoft's Clean Architecture guidance specifically describes infrastructure and UI depending toward the application core rather than the core depending outward.

25. Runtime Dependencies Are Different

At compile time:

```
EXAMPLE / FLOW
Application
    ↓
IGiftRepository
```

At runtime:

```
EXAMPLE / FLOW
CreateGiftService
    ↓
GiftRepository
    ↓
FundraisingDbContext
    ↓
SQL Server
```

DI connects the abstraction to the infrastructure implementation:

```
C#
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();
```

This is why:

```
EXAMPLE / FLOW
compile-time dependency direction
```

and:

```
EXAMPLE / FLOW
runtime call direction
```

must not be confused.

Diagram 3 — Fundraising Clean Architecture Flow

```
EXAMPLE / FLOW
HTTP Client
    ↓
Fundraising.Api
    ↓
CreateGiftService
    ↓
Gift Domain Model
    ↓
IGiftRepository
    ↑
GiftRepository
    ↓
FundraisingDbContext
```

```
    ↓
SQL Server
```

Compile-time core dependency:

EXAMPLE / FLOW

```
Infrastructure ———→ Application ———→ Domain
API ————————————→ Application
```

Runtime execution can still move outward through injected implementations.

26. Domain vs Application Logic

This is one of the most important architecture distinctions.

Domain logic

A rule is domain logic when the business itself cares about it.

Examples:

EXAMPLE / FLOW

```
Gift amount must be positive.

Gift allocations cannot exceed total Gift amount.

Completed Gift cannot be completed again.

A Fund must be active before receiving an allocation.
```

These rules belong near:

EXAMPLE / FLOW

```
Gift
GiftAllocation
Fund
Domain Service
```

27. Application Logic

Application logic coordinates a use case.

Examples:

EXAMPLE / FLOW

```
Load Gift.

Check user's permission.

Call Gift.Complete().

Persist changes.

Create OutboxMessage.

Commit transaction.
```

The Application layer decides:

EXAMPLE / FLOW

```
what steps form this use case
```

while the Domain decides:

EXAMPLE / FLOW

```
what business states/actions are valid
```

28. Infrastructure Logic

Infrastructure logic answers technical questions.

Examples:

EXAMPLE / FLOW

```
How do we load Gift from SQL Server?

How do we send email?

How do we publish to Azure Service Bus?

How do we store a PDF?

How do we call the payment provider?
```

These are important but not core business rules.

29. Presentation Logic

Presentation/API logic answers:

EXAMPLE / FLOW

```
Which route?

Which HTTP verb?

What status code?

Which request model?

Which response DTO?

How do validation errors become ProblemDetails?
```

This is presentation responsibility.

30. Keep Domain Entities Away From API Contracts

Avoid directly returning your EF/domain entity:

C#

```
return Ok(
    gift);
```

Why?

Your domain model may contain:

- Internal state

- Navigation properties
- Fields clients should not see
- Persistence-related relationships
- Methods
- Security-sensitive fields

API contracts should represent what the client is allowed and expected to consume.

Example:

```C#
public sealed record GiftResponse(
    long GiftId,
    decimal TotalAmount,
    string Status);
```

Microsoft's DDD guidance similarly warns against propagating domain entities into presentation areas where they do not belong.

31. Clean Architecture Is Not “Four Projects”

This misunderstanding is common.

You can create:

```EXAMPLE / FLOW
Domain
Application
Infrastructure
API
```

and still violate Clean Architecture completely.

For example:

```C#
// Domain project
using Microsoft.EntityFrameworkCore;

public sealed class Gift
{
    public FundraisingDbContext DbContext { get; }
}
```

The folder/project names do not matter if dependency direction is wrong.

Clean Architecture is primarily about:

```EXAMPLE / FLOW
dependency rules
responsibility boundaries
business independence
```

not solution decoration.

32. Clean Architecture Is Not Mandatory Everywhere

A simple CRUD service might not need:

EXAMPLE / FLOW

```
Domain
Application
Infrastructure
API
20 interfaces
30 handlers
```

Microsoft's architecture guidance explicitly notes that simpler data-driven services can use simpler approaches while richer DDD-style layering is most valuable for complex business domains.

A simple administration CRUD API might reasonably use:

EXAMPLE / FLOW

```
Controller
    ↓
DbContext
```

with appropriate validation and boundaries.

Architecture complexity should match business complexity.

33. Vertical Slice Architecture

Traditional layering organizes code by technical role:

EXAMPLE / FLOW

```
Controllers/
Services/
Repositories/
Validators/
DTOs/
```

Vertical Slice Architecture organizes code around:

EXAMPLE / FLOW

```
features/use cases
```

For example:

EXAMPLE / FLOW

```
Features/
    Gifts/
        Create/
        Complete/
        Refund/
        GetById/
        Search/

    Constituents/
        Create/
        UpdateAddress/
        Search/
```

A 2025 Microsoft .NET presentation specifically discussed Clean Architecture, Vertical Slices, and Modular Monoliths as complementary approaches rather than mutually exclusive choices.

34. Horizontal vs Vertical Organization

Horizontal organization:

EXAMPLE / FLOW

```
Controllers/
    GiftController

Services/
    GiftService

Validators/
    GiftValidator

Repositories/
    GiftRepository
```

One Gift feature may require editing five directories.

Vertical organization:

EXAMPLE / FLOW

```
Gifts/
    CreateGift/
        Endpoint.cs
        Command.cs
        Handler.cs
        Validator.cs
        Response.cs
```

Feature-related code stays near the feature.

35. Vertical Slice Does Not Eliminate Architecture

A vertical slice can still respect:

EXAMPLE / FLOW

```
Domain rules
Infrastructure boundaries
Dependency inversion
Security boundaries
```

For example:

EXAMPLE / FLOW

```
Application/
    Gifts/
        Create/
            CreateGiftCommand.cs
            CreateGiftHandler.cs

        Complete/
            CompleteGiftCommand.cs
            CompleteGiftHandler.cs
```

while:

EXAMPLE / FLOW

```
Domain/
    Gift.cs
```

still contains core Gift invariants.

36. Clean Architecture + Vertical Slices

APPLICATION ARCHITECTURE

These approaches answer different questions.

Clean Architecture asks:

> ⓘ Which dependencies are allowed?

Vertical Slices ask:

> ⓘ How should use-case code be organized?

They can work together.

Example:

EXAMPLE / FLOW

```
Fundraising.Domain/
    Gifts/
        Gift.cs
        GiftAllocation.cs

Fundraising.Application/
    Gifts/
        Create/
        Complete/
        Refund/

Fundraising.Infrastructure/
    Gifts/
        GiftRepository.cs

Fundraising.Api/
    Gifts/
        GiftEndpoints.cs
```

This can provide both:

EXAMPLE / FLOW

```
strong dependency direction
+
feature locality
```

37. Modular Monolith

A Modular Monolith is one deployment unit whose internal code is divided into strongly isolated business modules.

For the Fundraising Platform, modules might eventually include:

EXAMPLE / FLOW

```
Constituents

Fundraising

Engagement

Integrations
```

```
Administration
```

A module could own:

EXAMPLE / FLOW

```
Application use cases
Domain model
Persistence configuration
Contracts
```

while still running in one process.

38. Why a Modular Monolith Is Powerful

A Modular Monolith provides many benefits of explicit boundaries without immediately paying distributed-system costs.

You retain:

EXAMPLE / FLOW

```
one deployment
one debugger
simple transactions
easy refactoring
simple local development
```

while introducing:

EXAMPLE / FLOW

```
business module boundaries
controlled dependencies
clear ownership
```

This is often an excellent intermediate or long-term architecture.

39. Module Boundary Example

Suppose:

EXAMPLE / FLOW

```
Fundraising Module
```

owns:

EXAMPLE / FLOW

```
Gift
Campaign
Fund
GiftAllocation
```

while:

EXAMPLE / FLOW

```
Constituent Module
```

owns:

EXAMPLE / FLOW

```
Constituent
Address
ConstituentAddress
```

Fundraising should not reach directly into Constituents' internal tables/classes everywhere.

Instead it can depend on an explicit contract:

C#

```
public interface IConstituentLookup
{
    Task<ConstituentSummary?> FindAsync(
        long tenantId,
        long constituentId,
        CancellationToken cancellationToken);
}
```

This begins to create a genuine module boundary.

40. Module Boundaries Should Be Business-Oriented

Poor modules:

EXAMPLE / FLOW

```
Controllers Module
Repositories Module
DTO Module
Utilities Module
```

Those are technical categories.

Better modules:

EXAMPLE / FLOW

```
Constituents
Fundraising
Engagement
Payments
Receipting
```

because these represent business capabilities.

This aligns with the same domain-oriented thinking Microsoft recommends when identifying bounded contexts and microservice boundaries.

41. Modular Monolith vs Layered Monolith

Layered monolith:

EXAMPLE / FLOW

```
API
Application
Domain
Infrastructure
```

organized primarily by technical responsibility.

Modular monolith:

EXAMPLE / FLOW

```
Constituents
Fundraising
Engagement
Integrations
```

organized primarily by business capability.

You can combine them.

For example:

EXAMPLE / FLOW

```
Modules/
    Fundraising/
        Domain/
        Application/
        Infrastructure/

    Constituents/
        Domain/
        Application/
        Infrastructure/
```

42. Microservices Architecture

A microservices architecture divides a system into independently deployable services aligned to business capabilities/bounded contexts.

Microsoft's current Azure architecture guidance emphasizes that a microservice should be autonomous, business-oriented, independently deployable, and ideally own its data.

Example future decomposition:

EXAMPLE / FLOW

```
Constituent Service

Fundraising Service

Payment Service

Receipt Service

Engagement Service
```

But splitting by noun alone does not guarantee good boundaries.

43. Microservices Are Not “Small APIs”

A service with:

EXAMPLE / FLOW

```
Gift Service
Campaign Service
Fund Service
Allocation Service
```

might create a highly coupled distributed object model.

Completing one Gift might require:

EXAMPLE / FLOW

```
Gift Service
   ↓ HTTP
Campaign Service
   ↓ HTTP
Fund Service
   ↓ HTTP
Allocation Service
```

Now a simple business transaction became a distributed workflow.

That is often worse than the monolith.

Microsoft warns against overly granular services and chatty communication because these are signs of poor cohesion and excessive coupling.

44. Microservice Boundary Should Reflect Business Cohesion

A better boundary might be:

EXAMPLE / FLOW

```
Fundraising Service

owns:

Gift
Campaign
Fund
GiftAllocation
```

because these concepts participate in one tightly connected business capability.

This is why:

EXAMPLE / FLOW

```
Bounded Context
```

matters more than:

EXAMPLE / FLOW

```
one service per table
```

Microsoft's DDD guidance recommends defining a cohesive domain model per bounded context/business microservice.

45. Benefits of Microservices

Potential benefits include:

- Independent deployment
- Independent scaling
- Team autonomy
- Technology flexibility
- Better fault isolation when designed properly
- Smaller independent code bases
- Independent data evolution

Microsoft lists independent scalability, failure isolation, small focused teams, smaller codebases, and private service-owned data among common benefits.

46. Costs of Microservices

APPLICATION ARCHITECTURE

Microservices introduce distributed-system problems:

- Network failures
- Latency
- Serialization
- Versioned contracts
- Authentication between services
- Eventual consistency
- Distributed tracing
- Retry design
- Duplicate message handling
- Ordering
- Service discovery
- Deployment orchestration
- Monitoring
- Distributed transactions
- Operational overhead

This is why Microsoft explicitly notes that microservice benefits come with increased complexity and that simpler monolithic deployment can be a better choice when those costs outweigh the benefits.

47. When Microservices May Be Justified

Strong indicators include:

- Independent teams own separate business capabilities.
- Parts of the system require very different scaling.
- Independent deployment is valuable.
- Clear bounded contexts exist.
- Failure isolation is important.
- Operational maturity is strong.
- The organization can manage distributed observability and deployment.

Weak reason:

> ⓘ “Microservices are modern.”

48. Start With a Modular Monolith?

For many business applications:

EXAMPLE / FLOW

```
Modular Monolith
```

can be a strong starting architecture.

It allows you to discover genuine boundaries before paying distributed-system costs.

If a module later needs independent deployment:

EXAMPLE / FLOW

```
well-defined module boundary
        ↓
potential service boundary
```

The extraction is not automatic, but a disciplined module is much easier to separate than tangled code.

49. Database Ownership in a Monolith

One monolith can reasonably use one SQL Server database.

Modules may have:

EXAMPLE / FLOW

```
different schemas
```

for logical ownership:

EXAMPLE / FLOW

```
constituent.*
fundraising.*
engagement.*
integration.*
```

This matches your FundraisingDb approach.

For example:

EXAMPLE / FLOW

```
constituent.Constituent

fundraising.Gift

fundraising.Campaign

fundraising.Fund

engagement.Interaction

integration.OutboxMessage
```

Logical schema boundaries can reinforce architecture even inside one database.

50. Database Ownership in Microservices

Microservices should generally own their data rather than multiple services directly sharing and modifying the same schema.

Microsoft's microservices guidance explicitly recommends private service-owned storage and warns that shared schemas create coupling.

Bad:

EXAMPLE / FLOW

```
Gift Service ─┐
Campaign Service ─┼→ same tables
Fund Service ─────┘
```

Now services cannot evolve independently.

51. Transactions in a Monolith

APPLICATION ARCHITECTURE

Suppose creating a Gift requires:

EXAMPLE / FLOW

```
Insert Gift

Insert GiftAllocation rows

Insert OutboxMessage
```

With one relational database and one `DbContext`, this can frequently happen in one local transaction.

That is powerful and simple.

52. Transactions Across Microservices

Suppose:

EXAMPLE / FLOW

```
Fundraising Service
```

and:

EXAMPLE / FLOW

```
Receipt Service
```

have separate databases.

You generally cannot rely on one ordinary EF transaction covering both autonomous services.

Instead you may use:

EXAMPLE / FLOW

```
local transaction
+
message
+
eventual consistency
```

This is a major architectural change.

53. Domain Events vs Integration Events

These concepts should not be confused.

A **domain event** communicates something that happened within the domain/application boundary.

Example:

C#

```
public sealed record GiftCompletedDomainEvent(
    long GiftId);
```

It may trigger in-process behaviors.

An **integration event** communicates committed state to another process/system.

Example:

```C#
public sealed record GiftCompletedIntegrationEvent(
    long GiftId,
    long TenantId,
    decimal Amount);
```

Microsoft distinguishes domain events as typically in-process notifications within a domain from integration events intended for asynchronous communication across bounded contexts/services.

54. Why Integration Events Must Follow Commit

Imagine:

```EXAMPLE / FLOW
Publish GiftCompleted
        ↓
Database Save fails
```

Other systems now believe a Gift exists/completed when your database does not.

Integration messages should represent **committed state**.

Microsoft explicitly notes that integration events should propagate committed transactions and should occur only after the relevant state has successfully persisted.

55. Outbox Pattern

This creates the classic dual-write problem:

```EXAMPLE / FLOW
Save SQL
+
Publish message
```

Either operation can fail independently.

A common solution is the transactional Outbox:

```EXAMPLE / FLOW
Transaction
    ↓
Save Gift
Save OutboxMessage
    ↓
Commit
```

Then:

```EXAMPLE / FLOW
Background Outbox Processor
    ↓
Read unpublished messages
    ↓
Publish
    ↓
Mark processed
```

Your existing:

EXAMPLE / FLOW

```
integration.OutboxMessage
```

fits naturally into this architecture.

56. Integration Flow

EXAMPLE / FLOW

```
Create/Complete Gift
        ↓
Application use case
        ↓
Domain state changes
        ↓
Save Gift + OutboxMessage
        ↓
Commit SQL transaction
        ↓
Background worker
        ↓
Publish GiftCompletedIntegrationEvent
        ↓
Other systems
```

This avoids publishing an event for an uncommitted Gift.

57. Eventual Consistency

Once independent services own separate databases:

EXAMPLE / FLOW

```
everything updated immediately
inside one ACID transaction
```

may no longer be possible.

Instead:

EXAMPLE / FLOW

```
Fundraising commits Gift
        ↓
publishes event
        ↓
Receipt service receives event later
        ↓
Receipt state updated
```

For some interval:

EXAMPLE / FLOW

```
systems disagree
```

That is eventual consistency.

Microsoft's event-based microservice guidance explicitly describes event-driven cross-service workflows as eventually consistent distributed actions.

58. Idempotency

Distributed messages may be delivered more than once.

Therefore a consumer should often be able to process:

EXAMPLE / FLOW

```
Message X
Message X again
```

without producing duplicate business effects.

For example:

EXAMPLE / FLOW

```
GiftCompleted event
```

must not accidentally create two receipts if the same message is retried.

Possible design:

EXAMPLE / FLOW

```
MessageId
    ↓
Inbox/ProcessedMessages
    ↓
Already processed?
        Yes → ignore
        No  → execute + record
```

Distributed architecture requires thinking about duplicates explicitly.

59. CQRS

CQRS means:

EXAMPLE / FLOW

```
Command Query Responsibility Segregation
```

At its simplest:

EXAMPLE / FLOW

```
Commands
    →
change state

Queries
    →
return data
```

Microsoft emphasizes that CQRS can be logical only; it does **not** require separate databases or Event Sourcing. A simplified CQRS implementation can use one database while separating command and query models.

60. Simple CQRS in Fundraising

Commands:

EXAMPLE / FLOW

```
CreateGiftCommand

CompleteGiftCommand

CancelGiftCommand
```

Queries:

EXAMPLE / FLOW

```
GetGiftByIdQuery

SearchGiftsQuery

GetCampaignPerformanceQuery
```

Write side:

EXAMPLE / FLOW

```
Domain entities
Business invariants
Repository
Transaction
```

Read side:

EXAMPLE / FLOW

```
Direct optimized projections
DTOs
EF Core/Dapper queries
No need to load rich domain aggregates
```

61. CQRS Does Not Require MediatR

This is important.

CQRS is an architectural pattern.

You can implement:

C#

```
public sealed class CreateGiftHandler
{
}
```

and inject it directly.

You do not need a mediator library merely to call it.

Similarly:

EXAMPLE / FLOW

```
MediatR
```

does not automatically mean:

EXAMPLE / FLOW

```
CQRS
```

A library choice and an architectural pattern are separate decisions.

62. CQRS Does Not Require Separate Databases

APPLICATION ARCHITECTURE

Simple version:

EXAMPLE / FLOW

```
SQL Server
    ↑       ↑
Commands   Queries
```

Advanced version:

EXAMPLE / FLOW

```
Write Database
      ↓
Events / projection
      ↓
Read Database
```

Only introduce physical separation if the benefits justify consistency and operational complexity.

Microsoft's CQRS guidance explicitly supports a simple two-model approach backed by the same data source.

63. Query Side Should Be Practical

For a read-only API:

C#

```
var gifts =
    await dbContext.Gifts
        .AsNoTracking()
        .Where(
            gift =>
                gift.TenantId == tenantId)
        .Select(
            gift =>
                new GiftListItem(
                    gift.GiftId,
                    gift.TotalAmount,
                    gift.Status))
        .ToListAsync(
            cancellationToken);
```

You do not always need:

EXAMPLE / FLOW

```
Repository
    ↓
Domain aggregate
    ↓
Mapper
    ↓
DTO
```

for a read model.

CQRS often allows queries to take the shortest appropriate read path while keeping command rules strict.

64. Vertical Slice + CQRS

These combine naturally.

Example:

EXAMPLE / FLOW

```
Application/
    Gifts/
        Create/
            CreateGiftCommand.cs
            CreateGiftHandler.cs

        Complete/
            CompleteGiftCommand.cs
            CompleteGiftHandler.cs

        GetById/
            GetGiftByIdQuery.cs
            GetGiftByIdHandler.cs
```

Each use case becomes a slice.

65. Background Processing Architecture

Some work should not run synchronously during the HTTP request.

Examples:

- Large exports
- Receipt generation
- Email processing
- Outbox publishing
- Batch imports
- Expensive reports

A common architecture is:

EXAMPLE / FLOW

```
Web
  ↓
Queue
  ↓
Worker
```

Microsoft identifies Web-Queue-Worker as a standard architecture style for web frontends with resource-intensive or background work decoupled through asynchronous messaging.

Diagram 4 — Web + Background Worker

EXAMPLE / FLOW

```
Client
  ↓
ASP.NET Core API
  ↓
Save Request / Queue Work
  ↓
Message Queue
  ↓
Background Worker
```

```
   ↓
Long-running processing
   ↓
SQL / Storage / External API
```

Benefits include:

EXAMPLE / FLOW

```
faster HTTP response
independent worker scaling
failure isolation
retry capability
```

66. Do Not Use Fire-and-Forget Request Tasks

Bad:

C#

```
_ =
    Task.Run(
        async () =>
        await GenerateGiftExportAsync());
```

inside a controller.

The request can end while scoped dependencies disappear.

Use an owned background processing architecture instead:

EXAMPLE / FLOW

```
Channel
HostedService
Queue
Message broker
```

depending on durability requirements.

67. In-Memory Queue vs Durable Queue

`Channel<T>`:

EXAMPLE / FLOW

```
same process
fast
simple
not durable across process failure
```

Message broker:

EXAMPLE / FLOW

```
separate infrastructure
durable messaging possibilities
cross-process
more operational complexity
```

Choose based on requirement.

If losing queued work during process restart is unacceptable:

EXAMPLE / FLOW

```
in-memory Channel alone
```

is insufficient.

68. Cross-Cutting Concerns

Typical cross-cutting concerns include:

- Logging
- Authentication
- Authorization
- Exception handling
- Metrics
- Tracing
- Validation
- Caching
- Resilience
- Rate limiting

Do not manually duplicate these in every service.

Use appropriate architectural mechanisms.

69. Middleware

ASP.NET Core middleware is appropriate for HTTP-wide concerns such as:

EXAMPLE / FLOW

```
Exception handling
Authentication
Authorization
Correlation
Request logging
Rate limiting
```

Example:

C#

```
app.UseExceptionHandler();

app.UseAuthentication();

app.UseAuthorization();

app.MapControllers();
```

Middleware belongs at the HTTP pipeline boundary, not inside Domain.

70. Decorators / Pipelines

Application-level cross-cutting behavior can use:

EXAMPLE / FLOW

```
Decorators
Pipeline behaviors
Interceptors
```

For example:

EXAMPLE / FLOW

```
Validation
    ↓
Authorization
    ↓
Transaction
    ↓
Logging
    ↓
CreateGiftHandler
```

Be careful not to hide essential business sequence behind an enormous invisible pipeline.

71. Dependency Injection as Composition Root

`Program.cs` should wire the application together.

Example:

C#

```
builder.Services
    .AddScoped<
        IGiftRepository,
        GiftRepository>();

builder.Services
    .AddScoped<
        IUnitOfWork,
        EfUnitOfWork>();

builder.Services
    .AddScoped<
        CreateGiftService>();

builder.Services
    .AddScoped<
        CompleteGiftService>();

builder.Services
    .AddHostedService<
        OutboxBackgroundService>();
```

This is construction/configuration.

It is not business logic.

72. Configuration Architecture

Keep configuration externalized:

EXAMPLE / FLOW

```
Connection strings
API endpoints
Timeouts
Feature flags
Credentials references
```

Do not bury environment-specific values inside Domain classes.

Use strongly typed Options where appropriate.

The Domain should understand:

EXAMPLE / FLOW

```
MajorGiftThreshold
```

only if that concept genuinely belongs to business policy.

It should not understand:

EXAMPLE / FLOW

```
appsettings.Production.json
```

73. Security Architecture

Security must exist at multiple boundaries.

Edge/API

EXAMPLE / FLOW

```
Authentication
Authorization
Rate limiting
Input size controls
TLS
```

Application

EXAMPLE / FLOW

```
Permission checks
Tenant boundaries
Use-case authorization
```

Domain

EXAMPLE / FLOW

```
Business invariants
```

Infrastructure

EXAMPLE / FLOW

```
Secret management
Database permissions
Transport security
Encryption
```

One security mechanism does not replace the others.

74. Multi-Tenant Architecture

The Fundraising Platform examples frequently contain:

EXAMPLE / FLOW

```
TenantId
```

Every data access path must preserve tenant isolation.

Unsafe:

```
C#
FindAsync(
    giftId)
```

if Gift IDs are not sufficient to establish tenant ownership.

Safer contract:

```
C#
Task<Gift?> FindAsync(
    long tenantId,
    long giftId,
    CancellationToken cancellationToken);
```

Architecture should make tenant scope difficult to accidentally omit.

Potential mechanisms include:

- Explicit tenant parameter
- Scoped tenant context
- EF query filters
- Authorization policies
- Database security techniques

Do not rely on one mechanism alone without understanding its bypass paths.

75. Do Not Put Tenant State in Singleton

Bad:

```
C#
public sealed class TenantContext
{
    public long TenantId
    {
        get;
        set;
    }
}
```

registered:

```
C#
AddSingleton<TenantContext>();
```

Multiple requests would share tenant state.

That can become a serious cross-tenant security problem.

Use request/scoped ownership.

76. Scalability

There are two main forms:

Scale up

Add resources to one instance:

EXAMPLE / FLOW

```
More CPU
More memory
```

Scale out

Run multiple instances:

EXAMPLE / FLOW

```
API #1
API #2
API #3
```

Microsoft's architecture guidance distinguishes scaling vertically and horizontally and notes that monolithic applications can still scale horizontally by replicating the application instance.

Microservices are not required merely to scale horizontally.

77. Stateless API Instances

To scale horizontally, avoid storing essential request/session business state only in one process.

Bad:

EXAMPLE / FLOW

```
API instance #1
    ↓
static Dictionary<UserId, ImportantBusinessState>
```

If the next request reaches instance #2:

EXAMPLE / FLOW

```
state is absent
```

Prefer durable/shared state where appropriate:

EXAMPLE / FLOW

```
Database
Distributed cache
Message store
```

78. Caching Architecture

Cache is not a layer that should indiscriminately sit everywhere.

Ask:

EXAMPLE / FLOW

```
What data is cached?

Who owns freshness?

How long may it be stale?

Is it tenant-scoped?

What happens on cache failure?

Is invalidation required?
```

```
Can stale data violate security/business rules?
```

APPLICATION ARCHITECTURE

Cache is a performance optimization, not the source of truth unless explicitly designed that way.

79. Performance and Architecture

Architecture decisions can dominate performance.

Examples:

EXAMPLE / FLOW

```
1 local method call
```

vs:

EXAMPLE / FLOW

```
5 network service calls
```

or:

EXAMPLE / FLOW

```
one SQL query
```

vs:

EXAMPLE / FLOW

```
repository repeatedly loads entities
    ↓
N+1 queries
```

Do not optimize only at language level.

Changing:

C#

```
foreach
```

to a micro-optimized loop means little if architecture performs 40 unnecessary network calls.

80. Chatty Microservices

Suppose Gift creation requires:

EXAMPLE / FLOW

```
GET Constituent Service

GET Campaign Service

GET Fund Service

POST Payment Service

POST Gift Service

POST Receipt Service
```

for one synchronous user operation.

Problems:

- Latency accumulates
- Availability multiplies
- More authentication
- More failure combinations
- Harder testing

Microsoft explicitly calls overly chatty communication a symptom of tight coupling and poor cohesion between microservices.

81. Resilience Architecture

External systems fail.

Design must expect:

EXAMPLE / FLOW

```
Timeout
Transient network failure
Rate limit
Dependency outage
Partial failure
Duplicate response
```

Common mechanisms:

- Timeout
- Retry
- Circuit breaker
- Bulkhead/concurrency limit
- Fallback where meaningful
- Queueing
- Idempotency

Do not retry every failure blindly.

For example:

EXAMPLE / FLOW

```
HTTP 400
```

usually does not become successful after five retries.

82. Observability

A distributed system without observability becomes extremely difficult to operate.

Important signals:

EXAMPLE / FLOW

```
Logs
Metrics
Traces
Health checks
```

Current Aspire service defaults, for example, provide opinionated setup for OpenTelemetry, health checks, HTTP resilience, and service discovery in distributed applications.

Architecture should plan observability from the beginning rather than adding logging after production failures.

83. Correlation

Suppose one request flows through:

EXAMPLE / FLOW

```
API
  ↓
Payment service
  ↓
Message broker
  ↓
Receipt worker
```

You need to connect those operations.

Common identifiers include:

EXAMPLE / FLOW

```
TraceId
CorrelationId
MessageId
Business identifier such as GiftId
```

Do not use logs that say only:

EXAMPLE / FLOW

```
"Error processing request."
```

You need enough context to reconstruct what happened.

84. Aspire and Distributed .NET Architecture

As of current .NET tooling, Aspire provides a code-first model for composing and observing distributed applications.

It can model:

- Services
- Databases
- Containers
- Queues
- Caches
- Frontends
- Dependencies

and provides development-time orchestration and observability features.

But Aspire is **not** your business architecture.

It does not decide:

EXAMPLE / FLOW

```
Where Gift belongs

What your bounded contexts are

Whether a feature should be a microservice
```

```
Which business invariants belong in Domain
```

APPLICATION ARCHITECTURE

Microsoft explicitly describes Aspire as an orchestration/observability layer rather than a replacement for your application framework or cloud runtime.

85. Testing Architecture

Architecture should make different kinds of testing possible.

Unit tests

Test:

EXAMPLE / FLOW

```
Gift
GiftAllocation
Domain services
Application rules
```

without real SQL or HTTP.

Integration tests

Test:

EXAMPLE / FLOW

```
GiftRepository
EF Core mappings
SQL behavior
HTTP adapters
```

with real or realistic infrastructure.

API tests

Test:

EXAMPLE / FLOW

```
Routes
Authentication
Model binding
HTTP responses
Middleware
```

End-to-end tests

Test a critical complete flow.

86. Architecture Dependency Tests

You can also test architecture itself.

For example, enforce:

EXAMPLE / FLOW

```
Domain must not reference Infrastructure.

Application must not reference Api.

Domain must not reference ASP.NET Core.
```

Architecture rules are much stronger when automated than when written only in documentation.

87. Domain Unit Test Example

C#

```
[Fact]
public void Complete_WhenGiftIsNotFullyAllocated_Throws()
{
    Gift gift =
        Gift.Create(
            tenantId:
                1,
            constituentId:
                10,
            totalAmount:
                100m);

    gift.AddAllocation(
        fundId:
            20,
        amount:
            50m);

    Assert.Throws<
        InvalidOperationException>(
            () =>
                gift.Complete());
}
```

No:

EXAMPLE / FLOW

```
SQL Server
HttpClient
Controller
```

required.

That is one benefit of keeping Domain independent.

88. Application Unit Test

C#

```
[Fact]
public async Task CreateAsync_AddsGift()
{
    FakeGiftRepository repository = new();

    FakeUnitOfWork unitOfWork = new();

    CreateGiftService service =
        new(
            repository,
            unitOfWork);

    CreateGiftCommand command =
        new(
            TenantId:
                1,
            ConstituentId:
                10,
```

```
            TotalAmount:
                100m,
            Allocations:
            [
                new GiftAllocationRequest(
                    FundId:
                        5,
                    Amount:
                        100m)
            ]);

    await service.CreateAsync(
        command,
        CancellationToken.None);

    Assert.Single(
        repository.Gifts);
}
```

Again, the test can focus on the use case rather than infrastructure.

89. Integration Test

Repository tests should usually test the actual persistence behavior rather than mocking EF internals.

Questions include:

EXAMPLE / FLOW

```
Does mapping work?

Are constraints correct?

Does transaction behavior work?

Does tenant filtering work?

Are indexes/query shapes appropriate?
```

That is an Infrastructure concern.

90. Common Architecture Mistake — Controller Owns Everything

Bad:

C#

```
[HttpPost]
public async Task<IActionResult> Create(
    CreateGiftRequest request)
{
    // Validate business rules.
    // Query DB.
    // Insert Gift.
    // Charge card.
    // Send email.
    // Publish event.
    // Log audit.
}
```

The controller now owns:

EXAMPLE / FLOW

```
Presentation
Application logic
Domain rules
Infrastructure
Integration
```

This becomes difficult to test and change.

91. Common Mistake — Repository Owns Business Rules

Bad:

C#

```
public Task SaveAsync(
    Gift gift)
{
    if (gift.TotalAmount > 10_000)
    {
        gift.MarkAsMajor();
    }

    // Save.
}
```

Persistence behavior should not silently determine core business state unless that responsibility is explicitly part of the domain design.

92. Common Mistake — Domain Depends on Infrastructure

Bad:

C#

```
public sealed class Gift
{
    public async Task SaveAsync(
        FundraisingDbContext context)
    {
    }
}
```

Now Domain depends on EF Core.

You can no longer reason about Gift independently of persistence.

93. Common Mistake — Infrastructure Types Leak Everywhere

Examples:

EXAMPLE / FLOW

```
DbSet<Gift> returned from Application

SqlConnection passed into Domain

HttpResponseMessage returned from business service

EF entity used as external API contract
```

Each leak increases coupling.

Convert external/infrastructure representations at boundaries.

94. Common Mistake — Abstraction for Every Class

Clean Architecture does not mean:

EXAMPLE / FLOW

```
IGiftService
GiftService

IGiftValidator
GiftValidator

IGiftMapper
GiftMapper

IGiftHelper
GiftHelper
```

for everything.

Create abstractions where they represent:

EXAMPLE / FLOW

```
real architectural boundary
replaceable capability
test seam
polymorphic role
```

not merely because:

EXAMPLE / FLOW

```
interfaces look architectural.
```

95. Common Mistake — Repository Around Every EF Method

A repository that simply exposes:

C#

```
GetAll()
GetById()
Add()
Update()
Delete()
```

for every database table may add no meaningful abstraction.

As Topic 33 noted, EF Core `DbContext` already embodies Repository/Unit-of-Work concepts, so custom repositories should provide real domain/application value rather than duplicate EF mechanically.

We'll examine this deeply in Topic 35.

96. Common Mistake — Microservices by Database Table

Bad:

EXAMPLE / FLOW

```
Gift Microservice
GiftAllocation Microservice
```

```
Fund Microservice
Campaign Microservice
```

if every Gift operation requires all four synchronously.

That is a distributed monolith.

97. Distributed Monolith

A distributed monolith has:

EXAMPLE / FLOW

```
microservice deployment complexity
+
monolithic coupling
```

Symptoms:

- Services must deploy together.
- Services share one database schema.
- One business operation requires many synchronous service calls.
- Schema changes require several services to update simultaneously.
- Services share implementation libraries heavily.

This is often worse than a properly structured monolith.

98. Common Mistake — Shared Database Across Autonomous Services

If two services both directly modify:

EXAMPLE / FLOW

```
fundraising.Gift
```

neither truly owns it.

Service autonomy becomes mostly cosmetic.

Use explicit contracts/events rather than uncontrolled cross-service table access.

99. Common Mistake — CQRS Everywhere

A basic CRUD reference table may not need:

EXAMPLE / FLOW

```
CreateXCommand
CreateXHandler
GetXQuery
GetXHandler
UpdateXCommand
UpdateXHandler
DeleteXCommand
DeleteXHandler
```

unless separation provides actual value.

Architecture should reduce complexity, not ritualize it.

100. Common Mistake — Domain Events for Everything

Bad:

EXAMPLE / FLOW

```
GiftNameChanged
    ↓
handler
    ↓
calls another handler
    ↓
raises another event
    ↓
six hidden side effects
```

Use events where decoupling genuinely helps.

Mandatory linear workflow is often clearer as explicit orchestration.

101. Common Mistake — One Shared “Common” Project

Over time:

EXAMPLE / FLOW

```
Common
Shared
Utilities
Core
```

can become dumping grounds containing unrelated dependencies.

Then every module references them.

Result:

EXAMPLE / FLOW

```
everything depends on Common
```

which silently destroys module boundaries.

Shared libraries should contain only genuinely stable shared abstractions.

102. Common Mistake — Generic Base Classes for Architecture

Example:

C#

```
BaseController<
    TEntity,
    TRepository,
    TDto,
    TValidator,
    TMapper>
```

may technically reduce code but can make behavior difficult to understand.

Prefer explicit architecture over excessive generic inheritance.

103. Common Mistake — Architecture Astronautics

Suppose the application has:

EXAMPLE / FLOW

```
3 endpoints
1 database table
1 developer
```

and architecture includes:

EXAMPLE / FLOW

```
API Gateway
Event Bus
Microservices
CQRS
Event Sourcing
Saga
Service Mesh
Four databases
15 projects
```

That is complexity without corresponding need.

KISS and YAGNI still apply.

104. Common Mistake — No Architecture at All

The opposite is also dangerous:

EXAMPLE / FLOW

```
Everything in controllers.
Everything references DbContext.
Business rules copied everywhere.
```

Architecture should match complexity.

The goal is neither:

EXAMPLE / FLOW

```
maximum architecture
```

nor:

EXAMPLE / FLOW

```
zero architecture
```

but:

EXAMPLE / FLOW

```
appropriate architecture
```

105. Recommended Fundraising Platform Starting Architecture

For your learning project, a strong starting architecture would be a **well-structured modular/layered monolith**, not automatically microservices.

A possible solution:

EXAMPLE / FLOW

```
src/

    Fundraising.Api/

    Fundraising.Application/

    Fundraising.Domain/

    Fundraising.Infrastructure/

tests/

    Fundraising.Domain.Tests/

    Fundraising.Application.Tests/

    Fundraising.Infrastructure.Tests/

    Fundraising.Api.IntegrationTests/
```

This keeps deployment simple while providing meaningful boundaries.

106. Domain Organization

EXAMPLE / FLOW

```
Fundraising.Domain/

    Constituents/
        Constituent.cs
        Address.cs

    Gifts/
        Gift.cs
        GiftAllocation.cs
        GiftStatus.cs

    Campaigns/
        Campaign.cs

    Funds/
        Fund.cs

    Engagement/
        Interaction.cs
        TaskItem.cs
```

Alternatively, if modules become more independent, you might organize around larger business modules.

Do not obsess over the exact folder layout before understanding boundaries.

107. Application Vertical Slices

EXAMPLE / FLOW

```
Fundraising.Application/

    Gifts/

        Create/
            CreateGiftCommand.cs
            CreateGiftHandler.cs
```

```
        CreateGiftResult.cs

    Complete/
        CompleteGiftCommand.cs
        CompleteGiftHandler.cs

    GetById/
        GetGiftByIdQuery.cs
        GetGiftByIdHandler.cs

    Search/
        SearchGiftsQuery.cs
        SearchGiftsHandler.cs

Constituents/

    Create/
    GetById/
    Search/
```

This keeps feature logic together.

108. Infrastructure Organization

EXAMPLE / FLOW

```
Fundraising.Infrastructure/

    Persistence/
        FundraisingDbContext.cs

        Configurations/
            GiftConfiguration.cs
            CampaignConfiguration.cs
            FundConfiguration.cs

        Repositories/
            GiftRepository.cs

        Migrations/

    Messaging/
        OutboxProcessor.cs

    Email/
        ReceiptSender.cs

    Payments/
        PaymentGatewayAdapter.cs
```

Infrastructure remains technology-focused.

109. API Organization

EXAMPLE / FLOW

```
Fundraising.Api/

    Controllers/
        GiftsController.cs
        ConstituentsController.cs

    Contracts/
```

```
        Gifts/
            CreateGiftRequest.cs
            GiftResponse.cs

    Middleware/

    Authorization/

    Program.cs
```

Or use feature folders if preferred:

EXAMPLE / FLOW

```
Features/
    Gifts/
    Constituents/
```

The important part is that HTTP concerns remain HTTP concerns.

110. End-to-End Create Gift Flow

Suppose the client sends:

EXAMPLE / FLOW

```
POST /api/gifts
```

Request:

C#

```
{
  "tenantId": 1,
  "constituentId": 100,
  "totalAmount": 150,
  "allocations": [
    {
      "fundId": 10,
      "amount": 100
    },
    {
      "fundId": 20,
      "amount": 50
    }
  ]
}
```

Execution:

EXAMPLE / FLOW

```
GiftController
    ↓
CreateGiftHandler
    ↓
Gift.Create()
    ↓
Gift.AddAllocation()
    ↓
IGiftRepository.AddAsync()
    ↓
GiftRepository
    ↓
FundraisingDbContext
```

```
    ↓
SQL Server
```

The responsibilities remain distinct.

111. Complete Transaction Example

Application handler:

```C#
public sealed class CompleteGiftHandler
{
    private readonly IGiftRepository _giftRepository;
    private readonly IOutboxWriter _outboxWriter;
    private readonly IUnitOfWork _unitOfWork;

    public CompleteGiftHandler(
        IGiftRepository giftRepository,
        IOutboxWriter outboxWriter,
        IUnitOfWork unitOfWork)
    {
        _giftRepository = giftRepository;
        _outboxWriter = outboxWriter;
        _unitOfWork = unitOfWork;
    }

    public async Task HandleAsync(
        CompleteGiftCommand command,
        CancellationToken cancellationToken)
    {
        Gift gift =
            await _giftRepository.FindAsync(
                command.TenantId,
                command.GiftId,
                cancellationToken)
            ??
            throw new GiftNotFoundException(
                command.GiftId);

        gift.Complete();

        OutboxMessage message =
            OutboxMessage.Create(
                type:
                    "GiftCompleted",
                payload:
                    new
                    {
                        gift.GiftId,
                        gift.TenantId,
                        gift.TotalAmount
                    });

        await _outboxWriter.AddAsync(
            message,
            cancellationToken);

        await _unitOfWork.SaveChangesAsync(
            cancellationToken);
    }
}
```

The transaction can persist:

EXAMPLE / FLOW

```
Gift change
+
OutboxMessage
```

APPLICATION ARCHITECTURE

together.

112. Background Outbox Architecture

C#

```
public sealed class OutboxBackgroundService :
    BackgroundService
{
    private readonly IServiceScopeFactory _scopeFactory;

    public OutboxBackgroundService(
        IServiceScopeFactory scopeFactory)
    {
        _scopeFactory = scopeFactory;
    }

    protected override async Task ExecuteAsync(
        CancellationToken stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await using AsyncServiceScope scope = _scopeFactory.CreateAsyncScope();

            OutboxProcessor processor =
                scope.ServiceProvider
                    .GetRequiredService<
                        OutboxProcessor>();

            await processor.ProcessAsync(
                stoppingToken);

            await Task.Delay(
                TimeSpan.FromSeconds(5),
                stoppingToken);
        }
    }
}
```

Architecturally:

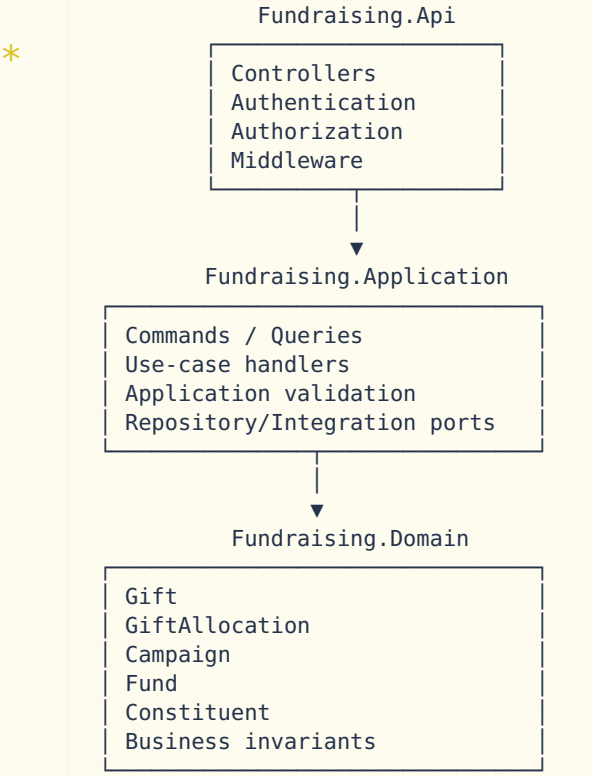
EXAMPLE / FLOW

```
API transaction
    ↓
Outbox table
    ↓
Worker
    ↓
External message broker
```

Diagram 5 — Complete Fundraising Architecture

EXAMPLE / FLOW

```
                        CLIENT
                          |
                          ▼
```




```
Infrastructure implements Application ports:

Fundraising.Infrastructure
        │
        ├── EF Core / SQL Server
        ├── Repositories
        ├── Payment adapters
        ├── Email
        ├── Messaging
        └── Outbox processor
```

113. Architecture Decision Guide

Use a simple monolith when:

EXAMPLE / FLOW

```
Business complexity is low
One team
Simple deployment
Limited scale requirements
```

Use a layered/Clean monolith when:

EXAMPLE / FLOW

```
Business rules are substantial
Infrastructure should be isolated
Testability matters
Codebase is growing
```

Use a modular monolith when:

EXAMPLE / FLOW

```
Several business capabilities exist
You need strong internal boundaries
One deployment remains advantageous
```

Consider microservices when:

EXAMPLE / FLOW

```
Boundaries are well understood
Independent deployment matters
Independent scaling matters
Teams can own services independently
Distributed-system complexity is justified
```

Use Web-Queue-Worker when:

EXAMPLE / FLOW

```
Some processing is long-running,
asynchronous, or independently scalable
```

Use CQRS when:

EXAMPLE / FLOW

```
Reads and writes have substantially
different models or complexity
```

Do not select them because they appear on an architecture diagram.

114. Architecture and Performance

A good architecture should make performance optimization possible without contaminating every layer.

Examples:

Read query

Use:

C#

```
.AsNoTracking()
.Select(...)
```

directly into a DTO.

Write workflow

Use:

EXAMPLE / FLOW

```
Domain entity
Repository
Transaction
```

Large export

Use:

EXAMPLE / FLOW

```
streaming
background worker
```

instead of loading everything into memory.

High-volume integration

Use:

```
EXAMPLE / FLOW
bounded concurrency
message queue
batching
```

Architecture determines which optimization opportunities exist.

115. Architecture and Memory

From Topic 30, object lifetime follows references.

Architecture can create memory problems when:

```
EXAMPLE / FLOW
Singleton
    ↓
unbounded cache
    ↓
large object graphs
```

or:

```
EXAMPLE / FLOW
background queue
    ↓
unbounded work items
```

or:

```
EXAMPLE / FLOW
long-lived DbContext
    ↓
thousands of tracked entities
```

Correct architectural lifetime boundaries directly improve memory behavior.

116. Architecture and Thread Safety

A layered architecture does not automatically make code thread-safe.

Examples:

```
EXAMPLE / FLOW
Singleton service
```

must still be safe for concurrent use.

```
EXAMPLE / FLOW
Scoped DbContext
```

must still not be used concurrently.

```
EXAMPLE / FLOW
Static cache
```

still requires correct concurrent collection/synchronization design.

Architecture determines ownership boundaries; Topic 28's concurrency ru

117. Architecture and Security

A strong architecture creates explicit trust boundaries.

For the Fundraising Platform:

EXAMPLE / FLOW

```
Client input
    ↓
API validation/authentication
    ↓
Application authorization
    ↓
Domain invariants
    ↓
Infrastructure data isolation
```

Each boundary has a different job.

Do not let:

EXAMPLE / FLOW

```
Controller authorization
```

be the only thing preventing a repository from returning another tenant's data.

Defense in depth is stronger.

118. Architecture and Failure

Ask what happens when every dependency fails.

For example:

EXAMPLE / FLOW

```
SQL unavailable?
Payment API timeout?
Message broker unavailable?
Email provider down?
Worker restarts?
Duplicate message delivered?
```

Good architecture defines:

EXAMPLE / FLOW

```
failure boundaries
retry policies
transaction boundaries
fallback behavior
durability
```

before production incidents force the decisions.

119. Architecture Documentation

Architecture should be understandable without reading every file.

Useful documentation includes:

EXAMPLE / FLOW

APPLICATION ARCHITECTURE

```
Context diagram

Container/deployment diagram

Module boundaries

Dependency rules

Important sequence diagrams

Data ownership

Integration contracts

Architecture Decision Records
```

Avoid documentation that becomes a 200-page document no one updates.

120. Architecture Decision Records

An ADR records a significant architectural decision.

Example:

EXAMPLE / FLOW

```
ADR-004: Use modular monolith rather than microservices

Context:
One team, shared domain, no independent scaling requirement.

Decision:
Deploy Fundraising Platform as one application with
module boundaries.

Consequences:
Simpler transactions and deployment.
Modules must not access each other's internal persistence directly.
```

This preserves the **reason** behind architecture.

121. Practical Exercise 1 — Layer Identification

Classify each component:

EXAMPLE / FLOW

```
GiftController
    →
Presentation/API

CreateGiftHandler
    →
Application

Gift
    →
Domain

GiftRepository
    →
Infrastructure
```

```
FundraisingDbContext
    →
Infrastructure

EmailReceiptSender
    →
Infrastructure
```


122. Practical Exercise 2 — Find Dependency Violation

Given:

```C#
// Fundraising.Domain
public sealed class Gift
{
    private readonly FundraisingDbContext _context;
}
```

Problem:

EXAMPLE / FLOW

```
Domain depends on persistence framework.
```

Refactor so persistence is handled outside the entity.

123. Practical Exercise 3 — Find Business Rule

Rule:

> **i** A Gift cannot be completed until allocations total the Gift amount.

Where should it live?

Best starting location:

EXAMPLE / FLOW

```
Gift domain model
```

because it is an invariant of Gift itself.

124. Practical Exercise 4 — Application Orchestration

Requirement:

EXAMPLE / FLOW

```
Load Gift
Complete Gift
Save changes
Write outbox message
```

Place orchestration in:

EXAMPLE / FLOW

```
Application handler/service
```

while:

EXAMPLE / FLOW

```
Gift.Complete()
```

owns the state-transition rule.

125. Practical Exercise 5 — Vertical Slice

Convert:

EXAMPLE / FLOW

```
Controllers/GiftController
Services/GiftService
Validators/GiftValidator
DTOs/GiftDto
```

into:

EXAMPLE / FLOW

```
Gifts/
    Create/
        CreateGiftEndpoint
        CreateGiftCommand
        CreateGiftHandler
        CreateGiftValidator
        CreateGiftResponse
```

while preserving Domain/Infrastructure boundaries.

126. Practical Exercise 6 — Modular Monolith

Identify possible modules:

EXAMPLE / FLOW

```
Constituent Management

Fundraising

Engagement

Integration
```

Define one public contract through which Fundraising can verify a Constituent exists without directly accessing every internal Constituent implementation class.

127. Practical Exercise 7 — Microservice Decision

Requirement:

EXAMPLE / FLOW

```
Receipt generation consumes heavy CPU
and scales independently from API traffic.
```

Possible options include:

EXAMPLE / FLOW

```
Background worker in same deployment

Separate worker deployment
```

```
Separate Receipt service
```

Do not automatically select microservice.

Compare:

EXAMPLE / FLOW

```
deployment independence
scale
failure isolation
data ownership
operational complexity
```

first.

128. Practical Exercise 8 — CQRS

Write side:

EXAMPLE / FLOW

```
CompleteGiftCommand
```

should load the appropriate domain object and enforce invariants.

Read side:

EXAMPLE / FLOW

```
GetGiftSummaryQuery
```

can project directly into:

C#

```
GiftSummaryDto
```

without reconstructing the entire domain aggregate when unnecessary.

129. Practical Exercise 9 — Outbox

Design transaction:

EXAMPLE / FLOW

```
Update Gift

Insert OutboxMessage

SaveChanges
```

Then design worker:

EXAMPLE / FLOW

```
Read pending message

Publish event

Record successful publication
```

Think about:

EXAMPLE / FLOW

```
duplicate delivery
worker crash
retry
message ordering
```

130. Practical Exercise 10 — Security Boundary

Find the problem:

C#

```
Task<Gift?>
    FindAsync(
        long giftId);
```

in a multi-tenant application.

Ask:

EXAMPLE / FLOW

```
Where is TenantId enforced?
```

If there is no guaranteed boundary, change the architecture rather than relying on every programmer remembering an extra `Where`.

131. Practical Exercise 11 — Distributed Failure

Suppose:

EXAMPLE / FLOW

```
Fundraising Service
    ↓
synchronously calls Receipt Service
```

Receipt Service is down.

Decide whether:

EXAMPLE / FLOW

```
Gift completion must fail

or

Gift can commit and receipt can be delivered later
```

That business decision determines whether synchronous integration or asynchronous messaging is appropriate.

132. Practical Exercise 12 — Architecture Review

For each component, answer:

EXAMPLE / FLOW

```
Who owns this?

What can depend on it?

What does it depend on?
```

```
What is its lifetime?

What data does it own?

Can it fail independently?

Does it require a transaction?

Can it be tested independently?

Would moving it to another process add real value?
```

APPLICATION ARCHITECTURE

These questions reveal architecture more effectively than asking only:

> ⓘ “Which pattern are we using?”

133. Exactly 25 Interview Questions

1. What is software/application architecture?

Application architecture is the high-level structure of a system: how responsibilities are divided, how components depend on and communicate with one another, where business rules live, how data is owned, and how the application is deployed and operated.

Architecture focuses on decisions that are expensive or important to change later, such as dependency direction, module boundaries, persistence boundaries, integration style, deployment topology, and consistency model.

2. What is the difference between a layer and a tier?

A **layer** is a logical separation of responsibilities within code.

A **tier** is a physical deployment boundary.

For example:

EXAMPLE / FLOW

```
API
Application
Domain
Infrastructure
```

can be four logical layers running inside one ASP.NET Core process.

If the web, business, and database components are deployed onto separate physical boundaries, they are also separate tiers.

Microsoft explicitly distinguishes logical layers from physical tiers in its architecture guidance.

3. What is a monolithic architecture?

A monolith is an application whose core behavior is deployed as one unit/process.

It can still contain:

EXAMPLE / FLOW

```
multiple projects
layers
modules
domain boundaries
```

and can still scale horizontally by running multiple copies.

Monolith does not mean poor architecture. Microsoft notes that well-struc

remain a suitable choice for many systems and often provide much simpler deployment and operations than microservices.

4. What is layered architecture?

Layered architecture separates software by technical responsibility, commonly into areas such as:

EXAMPLE / FLOW

```
Presentation
Business/Application
Data/Infrastructure
```

Each layer has a defined responsibility and allowed dependency relationships.

Traditional layering is straightforward but can cause business logic to depend on lower-level data-access implementations, which is one reason Dependency Inversion and Clean Architecture are often introduced.

5. What is Clean Architecture?

Clean Architecture organizes the system so core business/application logic is independent from infrastructure and presentation details.

Dependencies point inward toward the core.

Typical areas are:

EXAMPLE / FLOW

```
Domain
Application
Infrastructure
Presentation/API
```

Infrastructure implements abstractions required by the core instead of the core depending directly on EF Core, SQL Server, HTTP clients, or other implementation technology.

6. What is the dependency rule in Clean Architecture?

Source-code dependencies should point toward the business/application core.

For example:

EXAMPLE / FLOW

```
Infrastructure
    ↓
Application
    ↓
Domain
```

The Domain should not need a reference to Infrastructure.

At runtime, an Application service can still call an Infrastructure implementation through an injected interface.

Compile-time dependency direction and runtime call direction are therefore different concepts.

7. What belongs in the Domain layer?

The Domain layer contains business concepts and rules such as:

EXAMPLE / FLOW

```
Entities
Value objects
Aggregates
Business invariants
Domain services
Domain events
Business exceptions
```

For the Fundraising Platform that might include:

EXAMPLE / FLOW

```
Gift
GiftAllocation
Campaign
Fund
Constituent
```

The Domain should not directly depend on infrastructure frameworks such as EF Core or ASP.NET Core. Microsoft describes the domain model as the heart of business software and recommends keeping persistence details outside it.

8. What belongs in the Application layer?

The Application layer implements and orchestrates application use cases.

Examples:

EXAMPLE / FLOW

```
Create Gift
Complete Gift
Refund Gift
Search Gifts
```

It can coordinate repositories, authorization, transactions, integrations, and domain objects.

It should generally delegate business invariants to the Domain rather than becoming the primary location of domain rules.

9. What belongs in Infrastructure?

Infrastructure contains technical implementations such as:

EXAMPLE / FLOW

```
EF Core
DbContext
Repositories
SQL access
HTTP clients
Email providers
File storage
Message brokers
Distributed caches
```

Infrastructure implements abstractions required by Application/Domain.

Microsoft's Clean Architecture guidance places data access and other technology-specific implementations in the Infrastructure layer.

10. Why should domain entities not normally be returned directly from Web APIs?

APPLICATION ARCHITECTURE

Domain entities represent internal business state and behavior, not public transport contracts.

Returning them directly can expose:

EXAMPLE / FLOW

```
internal fields
navigation relationships
persistence details
security-sensitive information
unstable implementation structure
```

API-specific DTOs provide an explicit public contract and prevent domain/persistence design from becoming accidentally coupled to clients.

11. What is Vertical Slice Architecture?

Vertical Slice Architecture organizes application code around use cases/features instead of only technical layers.

For example:

EXAMPLE / FLOW

```
Gifts/
    Create/
    Complete/
    GetById/
```

rather than placing all controllers in one folder, all validators in another, and all handlers elsewhere.

Vertical slices can coexist with Clean Architecture: Clean Architecture controls dependency direction while vertical slicing improves feature locality. Microsoft has explicitly presented Clean Architecture, vertical slices, and modular monoliths as complementary architecture approaches.

12. What is a Modular Monolith?

A Modular Monolith is one deployable application containing strongly separated business modules.

For example:

EXAMPLE / FLOW

```
Constituents
Fundraising
Engagement
Integrations
```

may run in one process while exposing controlled module contracts.

It preserves monolithic operational simplicity while creating stronger business boundaries that can potentially support later extraction if independent deployment is eventually justified.

13. What is the difference between a modular monolith and microservices?

A modular monolith has:

EXAMPLE / FLOW

```
one primary deployment/process
```

with internal module boundaries.

Microservices have:

EXAMPLE / FLOW

```
multiple independently deployable processes/services
```

usually with independent data ownership and network-based communication.

Moving from modules to microservices adds distributed-system concerns such as network failures, eventual consistency, tracing, versioned contracts, retries, messaging, deployment coordination, and service discovery.

14. What is a bounded context?

A bounded context is a business/domain boundary within which a particular domain model and terminology are valid.

A microservice or module is often aligned to a bounded context.

For example, Fundraising might own:

EXAMPLE / FLOW

```
Gift
Campaign
Fund
GiftAllocation
```

rather than creating a microservice for every entity/table.

Microsoft recommends domain-oriented bounded contexts as the basis for microservice boundaries.

15. Why is a microservice per database table usually a bad design?

Tables do not necessarily represent independently deployable business capabilities.

If Gift, GiftAllocation, Fund, and Campaign are tightly related, splitting each into a separate service can turn one local business operation into several synchronous network calls.

That produces:

EXAMPLE / FLOW

```
latency
failure coupling
distributed transactions
complex deployment
```

and often creates a distributed monolith.

Microsoft warns against overly granular and chatty microservices because they indicate poor cohesion and excessive coupling.

16. What is CQRS?

CQRS stands for Command Query Responsibility Segregation.

It separates operations that:

EXAMPLE / FLOW

```
change state
```

from operations that:

EXAMPLE / FLOW

```
read state
```

A simple CQRS implementation can still use one database. Separate physical read/write databases and Event Sourcing are optional advanced choices, not requirements.

17. What is the difference between a command and a query?

A command expresses an intent to change state:

EXAMPLE / FLOW

```
CreateGift
CompleteGift
CancelGift
```

A query retrieves information without changing business state:

EXAMPLE / FLOW

```
GetGift
SearchGifts
GetCampaignSummary
```

Write-side models can enforce domain invariants, while query-side models can be optimized directly for the requested output.

18. What is the difference between a domain event and an integration event?

A domain event represents something that happened within a domain and is usually handled in-process.

An integration event communicates committed state to another bounded context, process, or external application.

Integration events therefore normally require durable asynchronous infrastructure and should be published only for successfully committed state.

19. What is eventual consistency?

Eventual consistency means independently updated components may temporarily contain different views of state but are expected to converge.

Example:

EXAMPLE / FLOW

```
Fundraising commits Gift
    ↓
publishes event
    ↓
Receipt service updates later
```

For a short time, Fundraising knows the Gift completed while Receipt has not yet processed it.

This is normal in many event-driven distributed systems.

20. What is the Outbox pattern and why is it useful?

The Outbox pattern addresses the dual-write problem:

EXAMPLE / FLOW

```
database commit
+
message publication
```

Instead of trying to make both happen independently, the application stores the business change and an Outbox message in one local database transaction.

A background process later publishes the Outbox record.

This significantly reduces the risk of committing business state but losing the integration message.

21. What is a distributed monolith?

A distributed monolith consists of multiple deployable services that remain tightly coupled.

Symptoms include:

EXAMPLE / FLOW

```
shared database
synchronized deployments
chatty synchronous calls
shared domain implementations
cross-service transactions
```

It pays the operational costs of microservices without obtaining meaningful autonomy.

22. When should you choose microservices?

Microservices become more reasonable when there are clear business boundaries and strong requirements for:

EXAMPLE / FLOW

```
independent deployment
independent scaling
team autonomy
failure isolation
independent technology/data evolution
```

and the organization can support distributed observability, deployment, security, messaging, and reliability.

Microsoft stresses both the benefits and the significant additional complexity of microservices.

23. How do architecture and dependency injection relate?

Architecture defines which components are allowed to depend on which abstractions.

DI composes the concrete runtime object graph.

For example:

EXAMPLE / FLOW

```
Application
    ↓
IGiftRepository
```

while startup registers:

C#

```
builder.Services
    .AddScoped<
```

```
        IGiftRepository,
        GiftRepository>();
```


Architecture determines the dependency direction.

DI merely supplies the runtime implementation.

24. How should you test a Clean Architecture application?

Use different test levels for different boundaries.

EXAMPLE / FLOW

```
Domain tests
    →
pure business rules

Application tests
    →
use-case orchestration with controlled dependencies

Infrastructure integration tests
    →
database/external implementations

API integration tests
    →
routing, authentication, middleware, HTTP contracts

End-to-end tests
    →
critical complete workflows
```

The independence of Domain and Application from infrastructure makes fast focused tests much easier.

25. What architecture would you choose initially for the Fundraising Platform?

A strong starting choice would be:

> **A modular, layered/Clean Architecture monolith with vertical slices inside the Application layer.**

A practical structure is:

EXAMPLE / FLOW

```
Fundraising.Api

Fundraising.Application

Fundraising.Domain

Fundraising.Infrastructure
```

with business-oriented feature organization such as:

EXAMPLE / FLOW

```
Gifts
Constituents
Campaigns
Funds
```

```
Engagement
Integrations
```

Use one SQL Server database initially with clear schema ownership:

EXAMPLE / FLOW

```
core
constituent
fundraising
engagement
integration
```

Use:

EXAMPLE / FLOW

```
Domain entities
    →
business invariants

Application handlers/services
    →
use-case orchestration

Infrastructure
    →
EF Core, SQL, email, messaging

API
    →
HTTP boundary
```

Add CQRS only where read/write separation improves the design, use `integration.OutboxMessage` for reliable external integration, and move a module into a separate service only when independent deployment/scaling/team ownership provides enough value to justify distributed-system complexity.

134. Quick Revision

Concept	Core idea
Architecture	High-level system organization
Layer	Logical responsibility boundary
Tier	Physical deployment boundary
Monolith	One deployable application
Layered Architecture	Separate technical responsibilities
Clean Architecture	Dependencies point toward core
Domain	Business concepts and invariants
Application	Use-case orchestration
Infrastructure	Technical implementations
API/Presentation	External HTTP boundary
Port	Core-facing abstraction
Adapter	Infrastructure implementation of port

Concept	Core idea
Vertical Slice	Organize by feature/use case
Modular Monolith	Strong modules, one deployment
Bounded Context	Boundary of one domain model
Microservice	Independently deployable business service
CQRS	Separate command and query models
Domain Event	In-process domain notification
Integration Event	Cross-process committed-state notification
Eventual Consistency	State converges asynchronously
Outbox	Reliable DB + messaging bridge
Web-Queue-Worker	Web and background processing split
Composition Root	DI registration/startup
Observability	Logs + metrics + traces
ADR	Record architectural decision and rationale

135. Final Architecture Decision Flow

Start with the business:

EXAMPLE / FLOW

```
Business requirements
        ↓
Non-functional requirements
        ↓
Identify business boundaries
        ↓
Identify changing dimensions
```

Then ask:

EXAMPLE / FLOW

```
Can one application satisfy the requirement?
```

If yes:

EXAMPLE / FLOW

```
Monolith
    ↓
Need stronger code boundaries?
    ↓
Layered / Clean Architecture
    ↓
Need strong business module isolation?
    ↓
Modular Monolith
```

Only then ask:

EXAMPLE / FLOW

```
Does a module genuinely require
independent deployment/scale/team ownership?
```

APPLICATION ARCHITECTURE

If yes:

EXAMPLE / FLOW

```
Candidate microservice
```

Then accept the consequences:

EXAMPLE / FLOW

```
Network communication
Independent data ownership
Retries
Timeouts
Observability
Eventual consistency
Messaging
Idempotency
Versioned contracts
Operational complexity
```

136. Final End-to-End Fundraising Concept Flow

EXAMPLE / FLOW

```
CLIENT
   ↓
HTTP / JSON
   ↓
Fundraising.Api
   ↓
Authentication + Authorization
   ↓
Request DTO
   ↓
Application Vertical Slice
   ↓
Create/Complete Gift Handler
   ↓
Domain Model
   ↓
Gift business invariants
   ↓
Application Port
   ↓
IGiftRepository
   ↓
Infrastructure Adapter
   ↓
GiftRepository
   ↓
EF Core DbContext
   ↓
SQL Server
```

For a transactional integration:

EXAMPLE / FLOW

```
Gift change
    +
OutboxMessage
    ↓
one database commit
    ↓
Outbox Worker
    ↓
Integration Event
    ↓
External system/service
```

For reads:

EXAMPLE / FLOW

```
HTTP Query
    ↓
Query Handler
    ↓
Optimized EF projection
    ↓
Read DTO
    ↓
HTTP Response
```

For background work:

EXAMPLE / FLOW

```
HTTP Request
    ↓
Queue durable work
    ↓
return response
    ↓
Worker
    ↓
Process asynchronously
```

For future growth:

EXAMPLE / FLOW

```
Modular Monolith
    ↓
Observe real boundaries
    ↓
One module needs independent deployment?
    ↓
Extract deliberately
    ↓
Microservice only when justified
```

The central architecture lesson is:

Architecture is the management of boundaries, dependencies, and change. Keep business rules independent of infrastructure, let Application coordinate use cases, keep Infrastructure at the edges, treat the API as an external boundary, organize growing systems around cohesive business capabilities, and introduce distributed architecture only when its operational cost is justified by real requirements.

For the Fundraising Platform specifically, the safest learning and production-oriented progression is:

EXAMPLE / FLOW

```
Well-structured monolith
        ↓
Clean dependency boundaries
        ↓
Vertical feature slices
        ↓
Business-oriented modules
        ↓
Reliable background/outbox integration
        ↓
Extract services only when
independent deployment truly matters
```

Next roadmap topic: Topic 35 — Repository, Unit of Work, Service Layer, and Mapping Patterns.